

simavr — A Detailed Tutorial

*Building, simulating, tracing, debugging, and
modern-AVR (AVRxt / ATtiny3217) support*

simavr project

June 06, 2026

Contents

1. What simavr Is (and Isn't)	4
2. Building simavr	5
2.1 Prerequisites	5
2.2 Compiling	5
2.3 Smoke test	5
3. Mental Model: How simavr Is Structured	6
3.1 The core: avr_t	6
3.2 Peripherals: avr_io_t	6
3.3 The IRQ mesh: avr_irq_t	6
3.4 Time: cycle timers	6
4. Your First Simulation	7
5. Telling simavr About the Firmware: the .mmcu Section	8
6. Connecting Virtual Hardware with IRQs	8
6.1 Watching a GPIO pin (firmware to your code)	8
6.2 Driving an input pin (your code to firmware)	8
6.3 Connecting two peripherals / parts	9
7. Cycle Timers	9
8. Waveform Tracing (VCD)	9
9. Debugging Firmware with GDB	10
10. Modern AVR (AVRxt) — the ATtiny3217	11
10.1 What's different about AVRxt	11
10.2 Modelled peripherals	11

10.3 Building and running modern firmware	11
10.4 Gotchas specific to modern AVR	12
11. Writing Your Own Test	12
12. The Example Boards	13
13. Quick API Reference	14
14. Troubleshooting	14
15. Where to Go Next	15

simavr — A Detailed Tutorial

simavr is a lean, mean and hackable AVR simulator written in C. It runs unmodified AVR firmware (the exact same `.elf` / `.hex` you flash to a real chip), models peripherals cycle-by-cycle, and exposes everything through a small, composable C API built around **IRQs** and **cycle timers**. This tutorial walks from a first build all the way to wiring up virtual hardware, tracing waveforms, debugging with GDB, and the newer **modern-AVR (AVRxt) / ATtiny3217** support.

1. What simavr Is (and Isn't)

simavr is a **library** (`libsimgavr`) first and a set of small example front-ends second. You link against it and drive the simulation from your own `main()`, or you use the bundled `run_avr` runner and the `examples/` boards.

What it does well:

- Executes real compiled AVR code instruction-by-instruction with accurate cycle counting.
- Models on-chip peripherals (timers, USART, SPI, TWI/I2C, ADC, EEPROM, GPIO) as pluggable modules.
- Lets you connect simulated external parts (LEDs, buttons, I2C/SPI devices, displays) to the chip's pins via an IRQ mesh.
- Produces **VCD** waveform traces you can open in GTKWave / PulseView / Surfer.
- Speaks the **GDB remote protocol** so you can single-step firmware in `gdb`.

What it is not: a SPICE-level analog simulator, nor a clock-accurate model of every undocumented silicon corner. It targets *functional* and *cycle* fidelity, which is what firmware development actually needs.

2. Building simavr

2.1 Prerequisites

On a Debian/Ubuntu/Kali-style system:

```
sudo apt install build-essential git \
    gcc-avr avr-libc gdb-avr \
    libelf-dev zlib1g-dev \
    freeglut3-dev                # only for the OpenGL examples
```

- `gcc-avr` / `avr-libc` build the *firmware* you will simulate.
- `libelf-dev` is required: `simavr` reads symbols and the `.mmcu` section from ELF files.
- The GLUT package is only needed for the graphical example boards.

2.2 Compiling

```
git clone https://github.com/buserror/simavr.git
cd simavr
make                # builds libsimavr, run_avr, and the tests/examples
```

Useful build knobs:

Variable	Effect
<code>RELEASE=1</code>	Optimised build (default is debug-friendly).
<code>make build-simavr</code>	Build just the core library and <code>run_avr</code> .
<code>V=1</code>	Verbose command echo.
<code>DESTDIR=...</code>	Staging directory for <code>make install</code> .

The library lands in `simavr/obj-<arch>/libsimavr.{a,so}` and the headers you need are under `simavr/sim/`.

2.3 Smoke test

```
cd tests
make run_tests        # runs the host-side regression suite
```

Each `test_*.c` builds a tiny AVR firmware, loads it, runs it, and asserts on the observed behaviour. They are the best worked examples of the API.

3. Mental Model: How simavr Is Structured

Four concepts carry almost everything.

3.1 The core: `avr_t`

`avr_t` is the CPU + memory + register state. It is produced by a **core descriptor** (one per supported device, e.g. `sim_mega328p.c`, `sim_tiny3217.c`). The descriptor declares flash/RAM sizes, the IO register map, vector table layout, fuses, and a per-device `init()` that wires up peripherals.

3.2 Peripherals: `avr_io_t`

Every peripheral (`avr_uart_t`, `avr_timer_t`, `avr_spi_t`) embeds an `avr_io_t`. It registers **read/write callbacks** on the IO addresses it owns and publishes a set of **IRQs**. Peripherals never poke each other directly; they talk through IRQs.

3.3 The IRQ mesh: `avr_irq_t`

An `avr_irq_t` is a single-value signal node. You can:

- **raise** it (`avr_raise_irq`) to push a new value,
- **register a callback** (`avr_irq_register_notify`) to be told when it changes,
- **connect** one IRQ to another (`avr_connect_irq`) so values flow automatically.

This is how a GPIO pin drives an LED, how a TWI master reaches a simulated EEPROM, and how the bundled parts in `examples/parts/` plug in.

3.4 Time: cycle timers

simavr does not step peripherals every cycle. Instead a peripheral schedules a callback N CPU cycles in the future with `avr_cycle_timer_register`. When the core reaches that cycle, the callback fires (and usually reschedules). This keeps long-idle timers free and makes the simulator fast.

```
firmware --writes IO--> peripheral callback
                        | schedules
                        v
cycle timer --fires--> raise IRQ --> your code / VCD
```

4. Your First Simulation

A minimal host program that loads an ELF and runs it:

```
#include <stdio.h>
#include "sim_avr.h"
#include "sim_elf.h"
#include "avr_ioport.h"

int main(int argc, char **argv)
{
    elf_firmware_t f = {0};
    if (elf_read_firmware(argv[1], &f)) {
        fprintf(stderr, "cannot read %s\n", argv[1]);
        return 1;
    }

    // The MCU name can come from the ELF's .mmcu section (see section 5)
    // or be forced explicitly:
    avr_t *avr = avr_make_mcu_by_name(f.mmcu[0] ? f.mmcu : "atmega328p");
    if (!avr) { fprintf(stderr, "unknown mcu\n"); return 1; }

    avr_init(avr);
    avr_load_firmware(avr, &f);          // sets flash, fuses, frequency, etc.

    for (;;) {
        int state = avr_run(avr);
        if (state == cpu_Done || state == cpu_Crashed)
            break;
    }
    return 0;
}
```

Build it against the library:

```
cc myrun.c -I simavr/simavr/sim \
-L simavr/simavr/obj-*/ -lsimavr -lelf -lm -o myrun
LD_LIBRARY_PATH=simavr/simavr/obj-*/ ./myrun firmware.elf
```

`avr_run()` executes a slice of instructions and returns the CPU state (`cpu_Running` , `cpu_Sleeping` , `cpu_Done` , `cpu_Crashed`). Your loop owns the pacing, so you can run flat-out, throttle to wall-clock, or stop on a condition.

5. Telling simavr About the Firmware: the `.mmcu` Section

Rather than pass MCU type and clock on the command line, embed them in the firmware with the `avr_mcu_section.h` macros. They emit a special `.mmcu` ELF section that simavr reads at load time.

```
#include "avr/avr_mcu_section.h"

AVR_MCU(8000000, "atmega328p");      // 8 MHz, part name
AVR_MCU_VOLTAGES(5000, 5000, 5000);  // VCC, AVCC, VREF in mV (optional)

// Auto-start a VCD trace of PORTB and the UART output:
const struct avr_mmcu_vcd_trace_t _trace[] _MMCU_ = {
    { AVR_MCU_VCD_SYMBOL("PORTB"), .what = (void*)&PORTB },
    { AVR_MCU_VCD_SYMBOL("UART"),   .mask = 0xff, .what = (void*)&UDR0 },
};
```

Now `run_avr firmware.elf` "just works": part, clock, and even the trace setup travel inside the binary. This is the idiom used throughout `tests/` and `examples/`.

6. Connecting Virtual Hardware with IRQs

6.1 Watching a GPIO pin (firmware to your code)

```
static void led_changed(avr_irq_t *irq, uint32_t value, void *param)
{
    printf("PB5 is now %s\n", value ? "ON" : "OFF");
}

avr_irq_t *pin = avr_io_getirq(avr, AVR_IOCTL_IOPORT_GETIRQ('B'), 5);
avr_irq_register_notify(pin, led_changed, NULL);
```

`AVR_IOCTL_IOPORT_GETIRQ('B')` selects PORTB; the index `5` is the pin. Each write the firmware makes to the pin raises this IRQ with the new level.

6.2 Driving an input pin (your code to firmware)

```
avr_irq_t *button = avr_io_getirq(avr, AVR_IOCTL_IOPORT_GETIRQ('D'), 2);
avr_raise_irq(button, 0); // press (active-low)
// ...later...
avr_raise_irq(button, 1); // release
```

6.3 Connecting two peripherals / parts

```
// Wire a simulated part's output straight to a chip input pin:
avr_connect_irq(part_output_irq,
                avr_io_getirq(avr, AVR_IOCTL_IOPORT_GETIRQ('C'), 0));
```

The `examples/parts/` directory has ready-made building blocks (`button.c`, `ledramp.c`, HD44780 LCD, I2C EEPROM, WS2812) that you connect this way. They are the canonical reference for writing your own parts.

7. Cycle Timers

Schedule something to happen later, in CPU-cycle time:

```
static avr_cycle_count_t every_ms(avr_t *avr, avr_cycle_count_t when, void *p)
{
    do_periodic_thing();
    return when + avr->frequency / 1000;    // reschedule 1 ms later
}

avr_cycle_timer_register(avr, avr->frequency / 1000, every_ms, NULL);
```

- Return `0` from the callback to stop; return an absolute cycle count to reschedule.
- Cancel early with `avr_cycle_timer_cancel(avr, every_ms, param)`.

Every timing-driven peripheral in `simavr` is built on exactly this primitive, including the modern timers covered in section 10.

8. Waveform Tracing (VCD)

VCD is the fastest way to see what firmware is doing.

```
#include "sim_vcd_file.h"

avr_vcd_t vcd;
avr_vcd_init(avr, "trace.vcd", &vcd, 100000 /* sample period, ns */);

avr_vcd_add_signal(&vcd,
    avr_io_getirq(avr, AVR_IOCTL_IOPORT_GETIRQ('B'), 5),
    1 /* bits */, "LED");

avr_vcd_start(&vcd);
// ... run the simulation ...
avr_vcd_stop(&vcd);
```

Open the result:

```
gtkwave trace.vcd      # or: pulseview / surfer trace.vcd
```

If you used the `.mmcu` trace macros from section 5, `simavr` starts and stops the VCD for you — no host code required.

9. Debugging Firmware with GDB

`simavr` embeds a GDB remote stub.

```
avr->gdb_port = 1234;
avr_gdb_init(avr);      // or set the GDB port and let run_avr do it
```

With `run_avr`:

```
run_avr -g firmware.elf      # -g waits for a debugger on :1234
```

Then, in another terminal:

```
avr-gdb firmware.elf
(gdb) target remote :1234
(gdb) break main
(gdb) continue
(gdb) info registers
```

You get breakpoints, watchpoints, single-stepping, and full memory/register inspection on the simulated core — using the real device's symbols.

10. Modern AVR (AVRxt) — the ATtiny3217

Classic AVRs (ATmega/ATtiny "AVRe") and the **modern** tinyAVR 0/1-series, megaAVR-0, and AVR Dx parts ("AVRxt") differ enough that simavr models them with a separate code path. This fork ships a complete **ATtiny3217** core (`simavr/cores/sim_tiny3217.c`).

10.1 What's different about AVRxt

Aspect	Classic (AVRe)	Modern (AVRxt)
Register/I/O layout	regs in data space	peripherals in extended I/O; flash mapped into data space
Config writes	direct	CCP unlock window for protected regs
Interrupts	global I-bit	CPUINT : NMI / LVL1 / LVL0, round-robin
Instruction timing	AVRe cycle counts	AVRxt timing (most stores/CALL faster)
Peripherals	TIMER0/1/2, USART	TCA/TCB/TCD, EVSYS, CCL, modern ADC/USART/SPI/TWI, NVMCTRL

The engine opts into all of this with arch flags (`AVR_ARCH_F_MODERN` | `AVR_ARCH_F_CCP` | `AVR_ARCH_F_XT_TIMING` | `AVR_ARCH_F_CPUINT`), set in `sim_core_declare_modern.h` .

10.2 Modelled peripherals

The ATtiny3217 core wires up: CLKCTRL, RSTCTRL, SLPCTRL, PORT/VPORT (A/B/C), TCA0, TCB0/1, TCD0, RTC+PIT, USART0, SPI0, TWI0, ADC0 (incl. temperature sensor with SIGROW calibration), DAC0, AC0, VREF, CCL, EVSYS, PORTMUX, NVMCTRL (EEPROM + flash self-programming), CRCSCAN, BOD/VLM, WDT, and SYSCFG/SIGROW device identity.

10.3 Building and running modern firmware

You need an avr-gcc new enough to know `-mmcu=attiny3217` (current avr-gcc / avr-libc with the ATtiny DFP). Then:

```
avr-gcc -mmcu=attiny3217 -DF_CPU=8000000 -Os blink.c -o blink.axf
```

In `tests/` the modern parts have dedicated targets (they are excluded from the default suite because they need that newer toolchain):

```
cd tests
make attiny3217-demo      # compiles + runs the blink demo
make attiny3217-selftest  # real firmware exercising many peripherals
```

A passing self-test looks like:

```
attiny3217 self-test on attiny3217 @ 8000000 Hz:
ok: EEPROM write/read (NVMCTRL)
ok: temperature sensor (SIGROW cal)
ok: TCB0 periodic counting
ok: TCA0 overflow counting
ok: DAC0 -> ADC0 internal channel
ok: EVSYS event -> ADC0 start
PASSED (6 subtests)
```

10.4 Gotchas specific to modern AVR

- **Protected writes** to CLKCTRL/NVMCTRL only take effect inside the CCP window. In firmware you do `CPU_CCP = CCP_IOREG_gc;` (or `CCP_SPM_gc`) right before the write; from host code use `avr_ccp_write(avr, AVR_CCP_IOREG)`.
- **EEPROM is mapped into the data space** below the IO end. It is persistent across resets (like real silicon) and is initialised to the erased state (`0xFF`) once at start-up.
- **Reset default clock is OSC20M / 6 (about 3.33 MHz).** `AVR_MCU(F_CPU, ...)` and the firmware's CLKCTRL setup decide the actual `CLK_PER`.
- **VPORT** registers (bit-addressable, low I/O) are aliases of the full PORT registers and are handled via an internal redirect — `sbi VPORTA_OUT, 0` works as expected.

11. Writing Your Own Test

The host-side test pattern (see `tests/test_avrxt_engine.c` for a large modern example) is:

```

avr_t *m = avr_make_mcu_by_name("attiny3217");
avr_init(m);
m->log = LOG_ERROR;

// Drive registers directly, or load a real firmware and run it:
avr_ccp_write(m, AVR_CCP_IOREG); // open the protected-write window
cpu_write(m, NVMCTRL_BASE + CTRLA, CMD_PAGEERASEWRITE);
for (int i = 0; i < 200; i++) avr_run(m); // let the busy phase complete

check("EEPROM committed", cpu_read(m, EE_BASE + 0), 0x5a);
avr_reset(m);
check("EEPROM persists across reset", cpu_read(m, EE_BASE + 0), 0x5a);

```

Add the source as `tests/test_<thing>.c`; the Makefile's `test_*.c` glob and the `run_tests` target pick it up automatically (modern-only firmware tests are gated behind explicit targets as noted above).

12. The Example Boards

`examples/` contains complete, runnable front-ends worth studying:

- `board_simduino` — an Arduino-compatible board with a virtual bootloader.
- `board_hd44780` — drives a character-LCD part from firmware.
- `board_ledramp`, `board_timer_64led` — GPIO/timer demos with OpenGL output.
- `parts/` — reusable simulated peripherals to drop into your own board.

Each board is a `main()` that makes a core, builds a few parts, connects IRQs, and runs the loop — i.e. everything in this tutorial, assembled.

13. Quick API Reference

Function	Purpose
<code>avr_make_mcu_by_name(name)</code>	Allocate a core by part name.
<code>avr_init(avr)</code>	Initialise core + peripherals (calls the device <code>init</code>).
<code>avr_load_firmware(avr,&f)</code>	Load flash/fuses/clock from an <code>elf_firmware_t</code> .
<code>elf_read_firmware(path,&f)</code>	Parse an ELF (incl. the <code>.mmcu</code> section).
<code>avr_run(avr)</code>	Execute a slice; returns the CPU state.
<code>avr_reset(avr)</code>	Reset the core (preserves persistent EEPROM on modern parts).
<code>avr_io_getirq(avr,iocctl,idx)</code>	Get a peripheral's IRQ node.
<code>avr_raise_irq(irq,value)</code>	Push a value onto an IRQ.
<code>avr_irq_register_notify(irq,cb,p)</code>	Subscribe to IRQ changes.
<code>avr_connect_irq(src,dst)</code>	Forward one IRQ to another.
<code>avr_cycle_timer_register(avr,when,cb,p)</code>	Schedule a future callback.
<code>avr_vcd_init / add_signal / start / stop</code>	Waveform tracing.
<code>avr_gdb_init(avr)</code>	Start the GDB remote stub.
<code>avr_ccp_write(avr,sig)</code>	Open the modern CCP protected-write window.

14. Troubleshooting

- **"unknown mcu"** — the part has no core descriptor, or the name in `.mmcu` doesn't match. Check `avr_make_mcu_by_name` against the cores in `simavr/cores/`.
- **Firmware seems to do nothing** — verify `F_CPU` in the firmware matches `AVR_MCU(...)`, and that you actually entered `main` (set a breakpoint via GDB).
- **libelf link errors** — install `libelf-dev` and link `-lelf`.

- **Modern firmware won't compile** — your avr-gcc/avr-libc predates the part; install a current ATtiny DFP or a newer toolchain.
 - **A protected register write "doesn't stick"** on a modern part — you forgot the CCP unlock immediately before it.
-

15. Where to Go Next

- Read a couple of `tests/test_*.c` — they are short, complete, and exercise one feature each.
- Read `examples/parts/` to learn the part-authoring pattern, then build your own board.
- For modern-AVR internals, see `doc/attiny3217_design.md` and the `simavr/sim/avr_*_modern.c` modern-peripheral sources.
- Upstream project: <https://github.com/buserror/simavr>

Happy simulating.